National University of Computer and Emerging Sciences

Lab Manual 04
Artificial Intelligence

Course Instructor	Ms. Umm-e-Ammarah
Lab Instructor	Muhammad Hasan
Lab Instructor Email ID	m.hasan@lhr.nu.edu.pk
Semester	Spring 2026

FAST - School of Computing
FAST-NU, Lahore Campus

Contents

Objectives:

- ✓ File Handling
  Read, Write, Append, Create
- ✓ OOP in Python
  Classes & Objects, Constructors, Instance vs Class Variables, Methods – Instance, Class & Static, Inheritance
- ✓ Blind Search Algorithms
  Breadth First Search (BFS), Depth First Search (DFS), Uniform Cost Search (UCS), Iterative Deepening Search (IDS)

Activities:

Activity 1 (Examples):

File Handling

File Handling - Opening Files

```
# file = open("filename", "mode")
# Modes: 'r', 'w', 'a', 'x'
# r -> Opens an existing file for reading - Default
# w -> Opens a new file for writing or overwrites an exiting file
# a -> Opens a file to add data at the end
# x -> Creates a new file. Fails if the file already exists

file = open('sample.txt', 'r')
print(file)
```

File Handling - Reading Files

```
file = open('sample.txt', 'r')
print(file,'\n')
print(file.read())

file = open('sample.txt', 'r')
content = file.read()
print(content)
print(len(content))
print(content.find('a'))

# readline()
with open('sample.txt', 'r') as file:
  while True:
    line = file.readline()
    if not line:
      break
    print(line.strip())
# readlines()
with open('sample.txt', 'r') as file:
  lines = file.readlines()
  print(lines)
```

```
# readlines() with for
with open('sample.txt', 'r') as file:
  for line in file.readlines():
    print(line.strip())

# reading line by line differently
file = open('sample.txt', 'r')

for line in file:
    print(line.strip())
```

File Handling - Creating Empty File

```
# Fails and throw error, if the file already exists
file = open('sample_test.txt', 'x')
print(file)
```

File Handling - Writing Files

```
# Append Mode - write() function
file = open('sample_test.txt', 'a')
print(file)

myString = 'This is a test text that I want to append to my file.\nThis
is new line.\n'
file.write(myString)

file.close()

# Write Mode - write() function
file = open('sample_test.txt', 'w')
print(file)

myString = 'This is a test text that I want to write to my file.\nThis
is new line.\n'
file.write(myString)

file.close()

# Write Mode - writelines() function
file = open('sample_test.txt', 'w')

myStrings = ['This is the first line.\n', 'This is the second line.\n',
'This is the third line.']
file.writelines(myStrings)

file.close()
```

File Handling - Closing Files

```
# Prevent resource leaks, slowing down the computer
# Unsaved data might not be written to the file properly
# Files might remain locked

file = open("example.txt", 'w')
file.write("Hello World!")
file.close()

with open("example2.txt", 'w') as file:
  file.write("Hello World!")
# No need to call file.close(), if file gets open using 'with' keyword
```

OOP in Python

Classes & Objects

```
class student:
  def __init__(self, name, age, grade):
    self.name = name
    self.age = age
    self.grade = grade

  def introduce(self):
    return f"My name is {self.name}, and I'm {self.age} years old."

# creating Objects
s1 = student("Ali", 20, "A")
s2 = student("Bilal", 21, "B")

# calling methods
print(s1.introduce())
print(s2.introduce())
```

 # Example

```
class car:
  def __init__(self, name, color, horse_power):
    self.name = name
    self.color = color
    self.horse_power = horse_power

  def honk(self):
    print(f"Car {self.name} is honking.")

c1 = car("Corolla", "black", 1800)
c2 = car("Civic", "white", 1500)
c3 = car("Swift", "black", 1300)
```

```
print(c1.name)
print(c1.color)
print(c1.horse_power)

c1.honk()
c2.honk()
c3.honk()
```

Constructors

```
class Dog:
  def __init__(self, name="unknown", breed="mixed"):
    self.name = name
    self.breed = breed

  def info(self):
    return f"{self.name} is a {self.breed}"

# creating objects with and without parameters
d1 = Dog("Tomy", "Labrador")
d2 = Dog()     # uses default values

print(d1.info())
print(d2.info())
```

 # Example

```
class car:
  def __init__(self, name='N/A', color='N/A', horse_power=0):
    self.name = name
    self.color = color
    self.horse_power = horse_power
    self.doors = 4

    print(f"Object created for {self.name}, {self.horse_power}")

  def honk(self):
    print(f"Car {self.name} is honking.")


c1 = car("Corolla", "black", 1800)
c2 = car("Civic", "white", 1500)
c3 = car("Swift", "black", 1300)
c4 = car()

print()
print(c1.name)
print(c1.color)
print(c1.horse_power)
print(c1.doors)
```

```
print()
c1.honk()
c2.honk()
c3.honk()
c4.honk()
```

Instance Variable - Unique to each object

```
class car:
  def __init__(self, brand, model, year):
    self.brand = brand  # Instance Variable
    self.model = model  # Instance Variable
    self.year = year    # Instance Variable

  def car_info(self):
    return f"{self.year} {self.brand} {self.model}"

# Creating Objects
car1 = car("Toyota", "Corolla", 2022)
car2 = car("Honda", "Civic", 2023)

# Assessing instance variables
print(car1.car_info())
print(car2.car_info())
```

Class Variable - same for all objects

```
class Dog:
  species = "Canine"  # Class Variable

  def __init__(self, name, age):
    self.name = name  # Instance Variable
    self.age = age    # Instance variable

# Creating Objects
dog1 = Dog("Buddy", 5)
dog2 = Dog("Charlie", 3)

# Accessing Variables
print(dog1.name, dog1.age, dog1.species)
print(dog2.name, dog2.age, dog2.species)
```

 # Example

```
class car:
  vehicle_type = "Motor Vehicle"
  def __init__(self, name='N/A', color='N/A', horse_power=0):
    self.name = name
    self.color = color
    self.horse_power = horse_power
```

```
    self.doors = 4

    print(f"Object created for {self.name}, {self.horse_power}")

  def honk(self):
    print(f"Car {self.name} is honking.")

c1 = car("Corolla", "black", 1000)
c2 = car("Civic", "white", 1500)
c3 = car("Swift", "black", 1300)
c4 = car()

print()
print(c1.name)
print(c2.name)
print(c3.name)
print(c4.name)

print()
print(c1.vehicle_type)
print(c2.vehicle_type)
print(c3.vehicle_type)
print(c4.vehicle_type)
```

Methods - Instance Methods

```
class car:
  def __init__(self, brand, model, year):
    self.brand = brand
    self.model = model
    self.year = year
    self.is_running = False   # Default State

  def start(self):
    self.is_running = True
    print(f"{self.brand} {self.model} has started.")

  def stop(self):
    self.is_running = False
    print(f"{self.brand} {self.model} has stopped.")

# Creating Objects
car1 = car("Toyota", "Corolla", 2022)


# Calling methods
car1.start()
car1.stop()
```

```python
# Example
class person:
  def __init__(self, name, age):
    self.name = name
    self.age = age

  def introduce(self):
    return f"Hi, I am {self.name}, and I am {self.age} years old."

p1 = person("Ali", 25)

print(p1.introduce())
```

Methods - Class Methods

```python
class Dog:
  species = "Canine"  # class variable

  @classmethod
  def change_species(cls, new_species):
    cls.species = new_species

print(Dog.species)
Dog.change_species("Wolf")
print(Dog.species)
```

Methods - Static Methods

```python
class MathOperations:
  @staticmethod
  def add(x, y):
    return x + y

print(MathOperations.add(5, 3))

# Example
class car:
  vehicle_type = "Motor Vehicle"

  def __init__(self, name='N/A', color='N/A', horse_power=0):
    self.name = name
    self.color = color
    self.horse_power = horse_power
    self.doors = 4
    self.speed = 0

  def honk(self):
    print(f"Car {self.name} is honking.")
```

```
  def accelerate(self, speed):
    self.speed = self.speed + speed

  @classmethod
  def change_vehicle_type(cls, vehicle_type):
    cls.vehicle_type = vehicle_type

  @staticmethod
  def add(x, y):
    return x + y

c1 = car("Corolla", "black", 1000)
c2 = car("Civic", "white", 1500)

print(c1.speed)

c1.accelerate(5)
print(c1.speed)

c1.accelerate(15)
print(c1.speed)


print(c1.vehicle_type)
print(c2.vehicle_type)

c1.change_vehicle_type("Hybrid Vehicle")

print(c1.vehicle_type)
print(c2.vehicle_type)

print(c1.add(4, 5))

# Example
class BankAccount:
  # Class variable
  bank_name = "National Bank"

  def __init__(self, name, balance):
    # Instance Variables
    self.name = name
    self.balance = balance

  # Instance Method
  def deposit(self, amount):
    if amount > 0:
      self.balance += amount
      print(f"{amount} deposited successfully.")
```

```
    else:
      print("Deposit amount must be positive.")

  # Instance Method
  def withdraw(self, amount):
    if amount > self.balance:
      print("Insufficient balance.")
    elif amount <= 0:
      print("Withdrawal amount must be positive.")
    else:
      self.balance -= amount
      print(f"{amount} withdrawn successfully.")

  # Static Method
  @staticmethod
  def validate_pin(pin):
    if len(pin) == 4 and pin.isdigit():
      return True

    return False

  # Class Method
  @classmethod
  def change_bank_name(cls, new_name):
    cls.bank_name = new_name
    print(f"Bank name changed to {cls.bank_name}")

# Create account
account1 = BankAccount("Hasan", 1000)

# Deposit money
account1.deposit(500)
print("Balance:", account1.balance)

# Withdraw money
account1.withdraw(300)
print("Balance:", account1.balance)

# Validate PIN
print("Valid PIN?", BankAccount.validate_pin("1234"))

# Check current bank name
print("Bank Name:", BankAccount.bank_name)

# Change bank name using class method
BankAccount.change_bank_name("State Bank")
# Check bank name again
print("Updated Bank Name:", BankAccount.bank_name)
```

Advanced OOP Concepts - Inheritance

```
class vehicle:

  def __init__(self, brand, speed):
    self.brand = brand
    self.speed = speed

  def move(self):
    return f"{self.brand} is moving at {self.speed} km/h."

class car(vehicle):
  def __init__(self, brand, speed, num_doors):
    super().__init__(brand, speed)
    self.num_doors = num_doors

  def honk(self):
    return "beep beep";

# creating an object
car1 = car("Toyota", 120, 4)

print(car1.move())
print(car1.honk())
```

 # Example

```
# Base Class
class Animal:
    def __init__(self, name):
        self.name = name
        print(f"Animal Constructor Called for {self.name}")

    def eat(self):
        print(f"{self.name} is eating.")

# Derived Class from Animal
class Dog(Animal):

    def __init__(self, name, breed):
        super().__init__(name)  # Call parent constructor
        self.breed = breed
        print(f"Dog Constructor Called for {self.name}")

    def bark(self):
        print(f"{self.name} is barking.")

# Derived Class from Animal
class Fish(Animal):
    def __init__(self, name, water_type):
```

```
        Animal.__init__(self, name)
        self.water_type = water_type
        print(f"Fish Constructor Called for {self.name}")

    def swim(self):
        print(f"{self.name} is swimming in {self.water_type} water.")

# Multilevel Inheritance (Dolphin -> Fish -> Animal)
class Dolphin(Fish):
    def __init__(self, name, water_type, intelligence_level):
        super().__init__(name, water_type)
        self.intelligence_level = intelligence_level
        print(f"Dolphin Constructor Called for {self.name}")

    def jump(self):
        print(f"{self.name} is jumping out of the water!")

# Creating Objects
print("----- Dog Object -----")
dog1 = Dog("Buddy", "Golden Retriever")

print("\n----- Fish Object -----")
fish1 = Fish("Nemo", "Fresh")

print("\n----- Dolphin Object -----")
dolphin1 = Dolphin("Flipper", "Salt", "High")

# Calling functions for each object
print("\n----- Dog Object Function -----")
dog1.eat()
dog1.bark()

print("\n----- Fish Object Functions -----")
fish1.eat()
fish1.swim()

print("\n----- Dolphin Object Functions -----")
dolphin1.eat()
dolphin1.swim()
dolphin1.jump()
```

 # Python Stacks

Stacks via List

```
# Stack follows LIFO (Last In First Out)

stack = []
```

```python
# push() operation -> append()
stack.append(10)
stack.append(20)
stack.append(30)

print("Stack after push operations:", stack)

# pop() operation -> removes last element
removed_item = stack.pop()
print("Popped item:", removed_item)

print("Stack after pop:", stack)

# peek (top element)
print("Top element:", stack[-1])

# check if stack is empty
if not stack:
    print("Stack is empty")
else:
    print("Stack is not empty")
```

 # Example – Stack Operations

```python
stack = []

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Exit")

    choice = input("Enter choice: ")

    if choice == '1':
        value = input("Enter value: ")
        stack.append(value)
        print("Stack:", stack)
    elif choice == '2':
        if len(stack) == 0:
            print("Stack Underflow! Stack is empty.")
        else:
            print("Removed:", stack.pop())
    elif choice == '3':
        if len(stack) == 0:
            print("Stack is empty.")
        else:
            print("Top element:", stack[-1])
```

```
    elif choice == '4':
        break
    else:
        print("Invalid Choice")
```

Stacks via collections.deque

```
# Stack using deque
from collections import deque

stack = deque()

# push
stack.append(100)
stack.append(200)
stack.append(300)

print("Stack:", stack)

# pop
print("Popped:", stack.pop())

print("Stack after pop:", stack)

# peek
print("Top element:", stack[-1])


# Why deque?
# Faster append and pop operations
# Efficient for large data
# O(1) time complexity for both ends
```

 # Python Queues

Queue using List

```
# Queue follows FIFO (First In First Out)

queue = []

# enqueue -> append()
queue.append("Ali")
queue.append("Bilal")
queue.append("Usman")

print("Queue:", queue)

# dequeue -> pop(0)
removed = queue.pop(0)
```

```
print("Removed:", removed)

print("Queue after dequeue:", queue)
# Note: pop(0) is slower (O(n)) because elements shift.

# Queue using collections.deque (Recommended)
from collections import deque

queue = deque()

# enqueue
queue.append("Ali")
queue.append("Bilal")
queue.append("Usman")

print("Queue:", queue)

# dequeue
print("Removed:", queue.popleft())

print("Queue after dequeue:", queue)

# peek (front element)
print("Front element:", queue[0])

# Example – Queue
from collections import deque


queue = deque()

while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Exit")

    choice = input("Enter choice: ")

    if choice == '1':
        value = input("Enter value: ")
        queue.append(value)
        print("Queue:", queue)
    elif choice == '2':
        if len(queue) == 0:
            print("Queue Underflow! Queue is empty.")
        else:
            print("Removed:", queue.popleft())
```

```
    elif choice == '3':
        if len(queue) == 0:
            print("Queue is empty.")
        else:
            print("Front element:", queue[0])

    elif choice == '4':
        break

    else:
        print("Invalid Choice")
```

 # Python Priority Queues

Priority Queue using heapq

```
import heapq

priority_queue = []

# insert elements
heapq.heappush(priority_queue, 30)
heapq.heappush(priority_queue, 10)
heapq.heappush(priority_queue, 50)
heapq.heappush(priority_queue, 20)

print("Priority Queue:", priority_queue)

# remove smallest element (highest priority)
print("Removed:", heapq.heappop(priority_queue))

print("Priority Queue after removal:", priority_queue)
# By default, heapq implements Min-Heap (smallest element first).
```

Priority Queue with Tuples (Priority + Value)

```
import heapq

tasks = []

# (priority, task)
heapq.heappush(tasks, (2, "Do Assignment"))
heapq.heappush(tasks, (1, "Attend Class"))
heapq.heappush(tasks, (3, "Watch TV"))
print("Tasks:", tasks)

while tasks:
    task = heapq.heappop(tasks)
    print("Processing:", task)
```

```
# Priority Queue using queue.PriorityQueue
from queue import PriorityQueue

pq = PriorityQueue()

pq.put((2, "Low Priority"))
pq.put((1, "High Priority"))
pq.put((3, "Very Low Priority"))

while not pq.empty():
    print(pq.get())

# In Python’s heapq and PriorityQueue,
# the element with the lowest value is removed first.
# Now, it's up to you, whether you assigned the highest priority task to
the lowest numnber
# or assigned the lowest priority task to the lowest numnber.
# In search algorithms such as Uniform Cost Search (UCS) and A*,
# priority queues are essential because nodes must be expanded based on
minimum path cost instead of arrival time.
# In Python, priority queues are most commonly implemented using the
heapq module.
```

Activity 2 (Blind Search Algorithms):

What Are Blind Search Algorithms?

Blind Search Algorithms (also called Uninformed Search Algorithms) are search techniques that do not use any additional knowledge about how close a state is to the goal.
They only know:

- Initial State
- Goal State
- Available Actions
- State Transitions

They do NOT know:

- Which state is closer to the goal
- Which path is better
- Any heuristic value

They explore the search space systematically.

State Space Representation

A State Space is the collection of all possible states of a problem and the transitions between them.
It can be represented as:

- Tree
- Graph

Each node = a state
Each edge = action / transition

What is a “State” in Python?

A State is any data structure that uniquely represents a situation.

Examples:

```
# Simple state
state = 'A'

# Coordinate state
state = (2, 3)

# Puzzle state
state = ((1,2,3),
         (4,5,6),
         (7,8,0))
```

Graph Representation in Python

Adjacency List (Unweighted Graph)

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}
```

Adjacency List with Cost (Weighted Graph)

```
graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 2)],
    'C': [('F', 4)],
    'D': [],
    'F': []
}
```

Node Representation

Search algorithms store nodes, not just states.
A Node contains:

- state
- parent
- depth
- path cost

Generic Node Class

```
class Node:
    def __init__(self, state, parent=None, cost=0, depth=0):
        self.state = state
        self.parent = parent
        self.cost = cost
        self.depth = depth
```

Path Reconstruction

```
def get_path(node):
    path = []
    while node:
        path.append(node.state)
        node = node.parent
    return path[::-1]
```

Types of Blind Search Algorithms

Breadth First Search (BFS)

- Explores level by level
- Uses a Queue (FIFO)
- Finds shortest path in unweighted graph
- Complete
- Optimal (for unweighted graphs)
- High memory usage

Algorithm

- Create queue
- Add start node
- While queue not empty:
  - Remove front node
  - If goal → return solution
  - Add unexplored neighbors

Python Implementation:

```
from collections import deque
def bfs(graph, start, goal):
    queue = deque([Node(start)])
    visited = set()

    while queue:
        current_node = queue.popleft()
        current_state = current_node.state
        if current_state == goal:
            return get_path(current_node)

        visited.add(current_state)

        for neighbor in graph[current_state]:
            if neighbor not in visited:
                child = Node(neighbor, current_node)
                queue.append(child)

    return None
```

Depth First Search (DFS)

- Explores as deep as possible
- Uses Stack or Recursion
- Memory efficient
- Not optimal
- May get stuck in infinite depth

Algorithm

- Push start node onto stack
- While stack not empty:
  - Pop top node
  - If goal → return
  - Push unexplored neighbors

Python Implementation

```
def dfs(graph, start, goal):
    stack = [Node(start)]
    visited = set()

    while stack:
        current_node = stack.pop()
        current_state = current_node.state

        if current_state == goal:
            return get_path(current_node)

        if current_state not in visited:
            visited.add(current_state)

            for neighbor in graph[current_state]:
                child = Node(neighbor, current_node)
                stack.append(child)

    return None
```

Uniform Cost Search (UCS)

- Expands node with lowest path cost
- Uses Priority Queue
- Works for weighted graphs
- Complete
- Optimal
- Expensive memory usage

Algorithm

- Push (0, start) into priority queue
- While queue not empty:

- Remove lowest cost node
- If goal → return
- Expand neighbors
- Update cost if cheaper

Python Implementation

```
import heapq

def ucs(graph, start, goal):
    pq = []
    heapq.heappush(pq, (0, Node(start)))
    visited = {}

    while pq:
        cost, current_node = heapq.heappop(pq)
        current_state = current_node.state

        if current_state == goal:
            return get_path(current_node)

        if current_state not in visited or cost < visited[current_state]:
            visited[current_state] = cost

            for neighbor, edge_cost in graph[current_state]:
                total_cost = cost + edge_cost
                child = Node(neighbor, current_node, total_cost)
                heapq.heappush(pq, (total_cost, child))

    return None
```

Iterative Deepening Search (IDS)

- IDS combines:
  - Memory efficiency of DFS
  - Optimality of BFS
- It performs DFS with increasing depth limit:
  - Depth = 0
  - Depth = 1
  - Depth = 2
  - ... until goal found
- Complete
- Optimal (unweighted graph)
- Medium memory

Depth Limited DFS

```
def depth_limited_dfs(node, graph, goal, limit):
    if node.state == goal:
        return node
```

```
    if node.depth >= limit:
        return None

    for neighbor in graph[node.state]:
        child = Node(neighbor, node, depth=node.depth + 1)
        result = depth_limited_dfs(child, graph, goal, limit)
        if result:
            return result

    return None
```

IDS Implementation

```
def ids(graph, start, goal, max_depth=10):
    for depth in range(max_depth):
        root = Node(start)
        result = depth_limited_dfs(root, graph, goal, depth)
        if result:
            return get_path(result)

    return None
```

Summary:

Algorithm	Data Structure	Optimal	Complete	Memory
BFS	Queue	Yes	Yes	High
DFS	Stack	No	No	Low
UCS	Priority Queue	Yes	Yes	High
IDS	Stack + Loop	Yes*	Yes	Medium

(*Optimal for unweighted graphs)

Activity 3 (Problem Statements / Tasks):

1. *Student Record Management System*

Create a Student Record System that stores student data in a file using OOP.

Requirements

Part A – OOP

- Create a class Student with:
  - Instance variables: name, age, grade
  - Instance method: introduce()
- Create a class variable:
  - school_name = "ABC School"
- Create a class method to change the school name.
- Create a static method:
  - Validate age (must be > 0).

Part B – File Handling

- Create a file students.txt using mode 'x'.
- Append at least 3 student records to the file using 'a'.
- Read the file:
  - Line by line
  - Print each line without extra spaces.
- Count total students in the file.
- Use find() to search if a particular name exists.

This problem statement/task should read/write file using objects created through Part A.

2. *Vehicle Management*

Design a vehicle hierarchy and demonstrate inheritance and method usage.

Requirements

Part A – Base Class

Create a class Vehicle with:

- Instance variables: brand, speed
- Method: move()

Part B – Derived Classes

- Create a class Car derived from Vehicle
  - Additional variable: num_doors
  - Method: honk()
- Create another derived class Bike
  - Additional variable: engine_capacity
  - Method: rev()

Part C – Multilevel Inheritance

Create a class ElectricCar derived from Car:

- Additional variable: battery_capacity
- Method: charge()

Part D – Demonstration

- Create objects of each class.
- Call all available methods.
- Show constructor chaining using super().

3. *Graph Search Simulator (BFS + DFS + IDS)*

Implement blind search algorithms on a given graph.

Given Graph (Unweighted)

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []       }
```

Requirements

- Implement:
  - BFS
  - DFS
  - IDS

- Use the provided Node class:

```
class Node:
    def __init__(self, state, parent=None, cost=0, depth=0):
        self.state = state
        self.parent = parent
        self.cost = cost
        self.depth = depth
```

- Reconstruct and print the path.
- Compare outputs:
  - Which path was found first?
  - Which algorithm uses more memory?

4. *Weighted Graph Search Using UCS (Priority Queue)*

Implement Uniform Cost Search on a weighted graph.
Given Graph

```
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2)],
    'C': [('D', 1)],
    'D': []
}
```

Requirements

- Implement UCS using heapq.
- Print:
  - Final path
  - Total cost
- Modify the graph so that:
  - One path becomes cheaper.
  - Observe the change in output.
- Explain why UCS differs from BFS.